

Wprowadzenie do programowania w R — cz. 1

Podstawy

Odwołanie do systemu pomocy

```
help("c")
```

lub prościej

```
?c
```

Odwołanie do systemu wyszukiwania w pomocy

```
help.search("vector")
```

lub prościej

```
??vector
```

Zmienne

Nazwa zmiennej, podobnie jak dowolny identyfikator w R (taki jak nazwa funkcji) może być dowolnym ciągiem znaków zawierającym litery, cyfry, znak podkreślenia i kropkę rozpoczynającym się od litery lub kropki, po której nie następuje cyfra.

Operatorem przypisania jest dwuznak `<-` (również w wersji działającej w prawo `->`) lub znak `=` (który działa tylko w lewo).

```
x <- 10  
2 -> y.z  
._e = 34.5  
x  
y.z  
._e
```

Funkcja `ls()` zwraca listę istniejących obiektów

```
ls()
```

a funkcja `rm()` je usuwa

```
rm(x)  
rm(list = ls())
```

Wektory numeryczne

Są to 2 pokrewne podstawowe typy danych w R, różniące się wewnętrzną reprezentacją: `double` i `integer`.

```
typeof(2)
mode(2)
```

```
typeof(2L)
mode(2L)
```

Wbrew pozorom literały `2` i `2L` nie oznaczają wartości skalarnych (nie ma takiego typu w R), tylko wektory o długości 1. Tworzenie (ręczne) wektorów o większej długości zwykle wykonuje się przy pomocy jednej z funkcji `c()`, `rep()`, `seq()` lub operatora `:`.

Funkcja `c()` — konkatencja wektorów

```
x <- c(2, 10, 17)
x
length(x)
y <- c(c(2, 3), 10, c(3, 2, -1))
y
length(y)
```

Funkcja `rep()` — powtarzanie elementów

```
z1 <- rep(x, times = 3)
z1
z2 <- rep(x, each = 3)
z2
z3 <- rep(x, length.out = 5)
z3
```

Funkcja `seq()` i operator `:` — ciągi arytmetyczne

```
2:10
seq(2, 10)
seq(from = 2, to = 10)
seq(to = 10, from = 2)
seq(from = 10, to = 2)
10:2
seq(2, 10, by = 2)
seq(2, 10, length.out = 8)
seq(2, 10, along.with = x)
```

`seq_along(x)` jest poręcznym (i szybszym) zamiennikiem `seq(along.with = x)`.

```
seq_along(x)
```

Operatory arytmetyczne i inne funkcje zwektoryzowane

Klasyczne funkcje matematyczne — w tym operatory arytmetyczne — są przykładami tzw. *funkcji zwektoryzowanych*. Są one “świadome” tego, że ich argumenty są wektorami i produkują wektorowy wynik.

```
x + c(1, 0, -1)
1:4 * 2:5
x^2
exp(y)
sqrt(seq(0, 1, by = 0.1))
```

W przypadku wektorów różnej długości stosowana jest tzw. **reguła odzysku** (*recycling rule*) — wektor krótszy jest wydłużany do długości dłuższego tak, jak gdyby była wywołana funkcja `rep(, length.out=)`. Jeśli długość dłuższego nie jest wielokrotnością długości krótszego, generowane jest ostrzeżenie.

```
x * 3 + 1
y + 1:4
```

Funkcje agregujące

Są to funkcje o argumentach wektorowych zwracające wartość skalarną, wśród nich podstawowe funkcje statystyki opisowej: `mean()`, `median()`, `var()`, `sd()`, `min()`, `max()`.

```
mean(x)
median(y)
var(rep(1, times = 20))
sd(2 * y - 3)
max(x)
which.min(x)
sum(x^2)
```

Wykorzystanie tych funkcji pozwala w wielu typowych sytuacjach uniknąć stosowania jawnych pętli.

Indeksowanie wektorów

Podstawowy sposób indeksowania wektorów w R przypomina indeksowanie tablic w językach typu C — z wyjątkiem tego, że indeksem pierwszego elementu jest 1.

```
x[1]
x[3]
```

Jednak operacja indeksowania jest ogólniejsza. Jak już wiemy, 1 jest wektorem. Wynik operacji również jest wektorem. Naturalnie więc można też indeksować dowolnym wektorem całkowitoliczbowym.

```
x[c(1, 3)]
y[2:4]
```

Dopuszczalne jest również indeksowanie liczbami całkowitymi ujemnymi. Oznacza to **brak** wskazanych składowych w wektorze wynikowym

```
x[-2]
y[-1:-3]
```

Niedopuszczalne jest mieszanie indeksów dodatnich i ujemnych.

Jak widać powyżej, operator `[]` zwraca *podwektor*. Istnieje też operator indeksowania `[][]` zwracający zawsze pojedynczy element (o ile taki istnieje).

```
x[[2]]  
y[[10]] # Ta instrukcja zwraca błąd
```

Wektory logiczne

Są to wektory o składowych przyjmujących jedną z wartości `TRUE` albo `FALSE`.

```
1 <- c(TRUE, FALSE, FALSE, TRUE, TRUE)  
1
```

Typowo powstają jako wynik operacji porównania wektorów numerycznych (w razie potrzeby działa reguła odzysku).

```
x == c(1, 2, 3)  
x > 10  
x * y^2 < 30
```

Na wektorach logicznych działają spójniki logiczne `&` (koniunkcja), `|` (alternatywa) i `!` (negacja).

```
y > 0 & y < 5
```

Koniunkcja i alternatywa mają wersję “podwójną” (`&&` i `||`), która uwzględnia tylko pierwsze składowe operandów, wykonuje skrócone wartościowanie i, w konsekwencji, produkuje zawsze wektor logiczny o długości 1.

```
y > 0 && y < 5
```

Często sensowniejszym sposobem uzyskania wektora logicznego o długości 1 jest skorzystanie z funkcji `all()` lub `any()`.

```
all(y > 0 & y < 5)  
any(y > 0 & y < 5)
```

Użyteczną cechą wektorów logicznych jest to, że mogą służyć do indeksowania innych wektorów.

```
y[c(TRUE, FALSE, FALSE, FALSE, FALSE, TRUE)]  
y[y > 0 & y < 5]  
y[y > median(y)]  
y[y < mean(y)]
```

Wartości logiczne w razie potrzeby konwertowane są na 0 i 1.

```
sum(y > 0)  
sum(y < mean(y))
```

Wektory znakowe (napisowe)

```
s <- c("Ala", "Ola", 'Ula')
s
s[2]
paste(s, collapse = "-")
paste(s, 1:6)
paste(s, 1:6, sep = "")
paste(s, 1:6, sep = "-", collapse = "===")
as.numeric("32")
as.logical("true")
```

Wartości nieistniejące

```
isFALSE(NA)
isTRUE(NA)
is.na(NA)
z <- c(1, 2)
z[5] <- 10
z
z + 30
z == 1
z == NA
is.na(z)
z[!is.na(z)]
mean(z)
mean(z[!is.na(z)])
mean(z, na.rm = TRUE)
z[is.na(z)] <- -1
z
```

[Porównaj działanie instrukcji `x[5]` i `x[[5]]`]

Wektory nazwane

Składowe wektorów mogą mieć nazwy

```
nx <- c(a = 2, b = -10, c = 0)
nx
```

W takiej sytuacji elementy mogą być indeksowane przy pomocy nazwy

```
nx['a']
nx[c("a", "b")]
nx[["c"]]
```

Nazwy elementów wektora dostępne są (do odczytu i zapisu) za pomocą *właściwości* `names` .

```
names(nx)
names(nx) <- c("pierwszy", "drugi", "trzeci")
nx
nx["drugi"]
```

Macierze

Macierze są wektorami o specjalnie ustawionych wymiarach, tzn. właściwości `dim`.

```
m <- matrix(c(1, 2, 3, 4), nrow = 2)
m
class(m)
dim(m)
m2 <- c(1, 2, 3, 4)
dim(m2) <- c(2, 2)
m2
class(m2)
dim(m2) <- c(1, 4)
m2
class(m2)
matrix(c(1, 2, 3, 4), ncol = 2, byrow = TRUE)
```

Operacje na macierzach

```
m <- matrix(1:4, nrow = 2)
m^2
sqrt(m) + matrix(rep(1, times = 4), nrow = 2)
m + 1
m < 2
m[1, 2]
m[2, 2] <- 0
m
m[2,]
m[,1]
m[1,] <- -1
m
```

Uwaga. Operator `*` jest mnożeniem po składowych (jak dla normalnych wektorów)

```
m1 <- matrix(1:4, nrow = 2)
m1
m2 <- diag(nrow = 2)
m2
m1 * m2
```

Jeśli chcemy pomnożyć macierze jak w algebrze liniowej, używamy operatora `%**%`

```
m1 %**% m2
```

Rozwiązanie układu równań $Ax = b$

```
A <- matrix(1:4, nrow = 2)
b <- c(1, 0)
solve(A, b)
```

Obliczenie macierzy transponowanej, odwrotnej, wyznacznik i ślad

```
t(A)
solve(A)
det(A)
sum(diag(A))
```

Instrukcje sterujące

W R mamy do dyspozycji standardowy zestaw instrukcji sterujących. Po pierwsze instrukcja warunkowa

```
if (runif(1) > 0.5) {
  "Orzeł"
}
```

również z klauzulą `else`

```
if (runif(1) > 0.5) {
  "Orzeł"
} else {
  "Reszka"
}
```

Funkcja `runif()` (`?runif`) zwraca jedną lub więcej liczb wylosowanych według jednostajnego rozkładu prawdopodobieństwa — domyślnie na przedziale $[0, 1]$.

Użyteczna bywa też funkcja `ifelse()`

```
x <- runif(20)
rzuty <- ifelse(x > 0.5, "Orzeł", "Reszka")
rzuty
```

Jest też pętla warunkowa

```
liczba_orlow <- 0
while (runif(1) > 0.5) {
  liczba_orlow <- liczba_orlow + 1
}
liczba_orlow
```

pętla (potencjalnie) nieskończona

```
liczba_orlow <- 0
repeat {
  if (runif(1) <= 0.5) break
  liczba_orlow <- liczba_orlow + 1
}
liczba_orlow
```

i pętla iteratorowa

```
x <- runif(20)
x
for (i in 1:length(x)) {
  print(mean(x[1:i]))
}
```

Alternatywą dla pętli (zwłaszcza `for`) są funkcje zwektoryzowane oraz funkcje wspierające styl funkcyjny, w szczególności rodzina funkcji `lapply()` (`?lapply`).

Funkcje

Funkcje w R są obiektami. Tworzy się je przy pomocy funkcji `function()`. Zwracaną wartością jest wartość ostatniego wyrażenia lub argument funkcji `return()`.

```
f <- function(x, a = 2) x^2 + 10 * a

g <- function(z) {
  stopifnot(all(z > 0))
  sin(z^2) - pi
}

h <- function(x) {
  if (x < 0) {
    return(x + 10)
  }
  cos(x - 1)
}
```

Wywołania funkcji

```
f(10)
f(10, 4)
f(a = -1, x = 3)
f(c(-1, 1))
g(3)
g(-3)
h(-1)
h(10)
```

Od wersji R 4.1.0 dostępny jest operator *potoku* [ang. *pipe*] `|>` pozwalający (formalnie) zastąpić zagnieżdżone wywołania funkcji łańcuchem tych wywołań, np.

```
# wywołania bez potoku
h(g(f(20)))
g(f(10, 3))

# są równoważne następującym konstrukcjom
20 |> f() |> g() |> h()
10 |> f(3) |> g()
```

Operator potoku jest alternatywą dla dostępnego również we wcześniejszych wersjach R podobnego operatora `%>%` zdefiniowanego w pakiecie `magrittr` i wykorzystywanego w wielu innych pakietach.

Rysowanie

Dane jednowymiarowe

Histogram

```
x <- sample(1:1000, 100, replace = TRUE)
hist(x)
hist(x, freq = FALSE)
hist(x, breaks = 4)
hist(x, breaks = c(0, 210, 650, 884, 1000))
```

Wykres pudełkowy

```
boxplot(x)
boxplot(x, horizontal = TRUE)
```

Dane dwuwymiarowe

Wykres rozproszenia (*scatter plot*)

```
x <- sample(1:1000, 100, replace = TRUE)
y <- sample(1:1000, 100, replace = TRUE)
plot(x, y)
plot(x, y, xlab = "Zmienna x", ylab = "Zmienna y", main = "Wykres y względem x")
```

Rysunek można zachować w pliku

```
pdf("scatterplot.pdf")
plot(x, y, col = "pink")
dev.off()
```

Wykres funkcji

```
x <- seq(-pi, pi, length.out = 100)
plot(x, sin(x), type = "l", ylab = "y")
lines(x, cos(x), type = "l", col = "orange")
```